

Technical disclosure - scalar interval thermal governor with an exact pending-command queue

Private working draft for technical review. No filing recommendation. Patentability is unresolved, with high overlap for broad formulations. No human inventor is assigned by this document.

Abstract

A discrete-time thermal governor maintains a finite interval for nonnegative temperature rise in a scalar model with bounded coefficients and disturbance. It intersects that interval with an observation set formed from a temperature sensor whose noise precedes upper clipping. A clipped observation supplies a lower constraint without imposing a finite measurement upper constraint. The governor predicts the upper temperature bound through an externally supplied exact FIFO of accepted pending efforts. It selects a requested-effort-limited grid action using the remaining thermal allowance and accepts the calculation only when the current state, queued transitions and candidate transition satisfy a temperature limit. A validated zero-input invariant set supplies the condition for an indefinite fallback tail. Invalid queue data, inconsistent observations and loss of this feasibility condition latch the governor invalid. An externally asserted applied effort advances the interval. The implementation is a synthetic scalar demonstrator with no hardware or network protocol.

Field, purpose, and status

[0001] This disclosure concerns a numerical governor for choosing normalized effort in a discrete-time scalar thermal model. The delivered embodiment is the `interval_queue` policy in `src/governor.py`, exercised by `run_experiments.py`. The code uses Python standard-library arithmetic and synthetic data. A decision is a per-step calculation, not a network authorization lease, an authenticated message, or a hardware safety certification.

[0002] The motivating error is to reduce effort based only on a sensor value or a newly proposed command while accepted prior commands remain pending. Clipping also prevents the displayed upper-limit value from establishing an upper bound on the protected state. The supplied research paper already discloses clipped high-temperature evidence and FDIR intervention; those premises are background. The present draft describes this implementation without asserting that combining its techniques is new.

[0003] The only safety statement developed here is conditional satisfaction of a scalar sampled-state inequality under explicit model, initialization, observation and command assumptions. The code does not identify thermal parameters, acquire real sensors, authenticate acknowledgments, detect every contract violation, model intermediate temperatures between sample instants, or model a hidden winding hotspot distinct from the scalar state. Human inventorship, entitlement and any filing decision remain unassigned and unresolved.

State and model

[0004] Let k count discrete model steps. Let T_a be a fixed ambient temperature in degrees Celsius, $x_k = T_k - T_a$ be nonnegative temperature rise in degrees Celsius, and u_k be a dimensionless applied effort in $[0, 1]$. The modeled state transition is

$$x_{k+1} = a_k x_k + b_k u_k^2 + w_k.$$

The dimensionless retained-rise coefficient a_k lies in $[a_{\min}, a_{\max}]$; b_k lies in $[b_{\min}, b_{\max}]$ and denotes temperature-rise contribution per model transition at unit effort; w_k lies in $[w_{\min}, w_{\max}]$ and denotes additive rise per model transition. The simulation assigns one second per transition, as specified in the experiment protocol. The constants are illustrative and must be reidentified for different real sampling intervals.

[0005] The model permits arbitrary coefficient/disturbance values inside these intervals at each transition. The simulator normally samples a and b once per episode and w at each step; this simulator choice is a subset of the bounded model assumption. Coefficient correlations are not exploited. For $x \geq 0$ and $u \geq 0$, monotonicity gives the bounding transition

$$L_{\text{next}} = a_{\min} L + b_{\min} u^2 + w_{\min},$$

$$H_{\text{next}} = a_{\max} H + b_{\max} u^2 + w_{\max}.$$

These equations are propagate. They are exact extrema of the stated rectangular parameter/state bounds in real arithmetic. The implementation uses ordinary floating-point arithmetic and tolerances, not directed-rounding interval arithmetic or a verified floating-point proof.

[0006] `Model.__post_init__` requires every model field to be finite; $0 \leq a_{\min} \leq a_{\max} < 1$; $0 \leq b_{\min} \leq b_{\max}$ with $b_{\max} > 0$; $0 \leq w_{\min} \leq w_{\max}$; $T_a < C < T_{\text{limit}}$; nonnegative observation error bound `epsilon`; and an action-grid step `Delta` in $(0, 1]$. Define the allowed rise $R = T_{\text{limit}} - T_a$. It additionally requires $w_{\max} / (1 - a_{\max}) \leq R$, which is used for the zero-input tail in [0017]. An invalid model raises `ValueError` before the governor is constructed.

[0007] Construction requires a commissioned interval $[L_0, H_0]$ satisfying $0 \leq L_0 \leq H_0 < \text{infinity}$. The code accepts this interval as an externally supplied assumption; it does not establish it from a clipped reading. Its validity means that the true initial scalar state lies within the interval. The constructor does not require $H_0 \leq R$; a subsequent observation can narrow the interval, but a decision whose resulting upper bound is above the limit cannot receive the described feasibility result.

Observation processing

[0008] The sensor assumption is $y_k = \min(T_a + x_k + v_k, C)$ with $|v_k| \leq \text{epsilon}$. Noise is applied **before** clipping. There is only an upper clipping operation. The `None` value represents an absent observation. A clipping level, error bound and ambient value have the same temperature units as the displayed reading.

[0009] For an unsaturated observation, the measurement-consistent rise interval is $[y_k - T_a - \text{epsilon}, y_k - T_a + \text{epsilon}]$. For an observation at the upper clipping level, it is the half-line $[C - T_a - \text{epsilon}, \text{infinity})$. The latter adds no finite upper constraint. For a missing observation there is no measurement intersection, and the prior prediction interval is retained.

[0010] `intersect_reading` intersects the prior $[L, H]$ with the applicable observation set and with the nonnegative state domain. In exact notation, $L_{\text{post}} = \max(L, 0, y - T_a - \text{epsilon})$ and $H_{\text{post}} = \min(H, U_y)$, where $U_y = y - T_a + \text{epsilon}$ if unsaturated and $U_y = \text{infinity}$ if clipped. An empty intersection raises `ValueError`; the caller latches the governor invalid. The code does not rebase its state to an inconsistent reading. It classifies readings within $1e-9$ of `C` as clipped, rejects nonfinite packets and readings more than $1e-9$ above `C`, and tolerates a corresponding small interval overlap discrepancy. Those tolerances qualify strict arithmetic statements in this draft.

Queue contract and candidate selection

[0011] At a decision, `pending` is an externally supplied exact FIFO $q = (q_0, \dots, q_{(d-1)})$ containing all accepted commands that will execute before the new candidate. q_0 is the next applied command. The delivered contract is synchronous and known; the code does not infer FIFO contents from packet histories. Empty `pending` means the candidate can apply on the current transition. The caller must preserve the ordering and inclusion of all committed commands.

[0012] `Governor.decide` requires finite demand r in $[0, 1]$; invalid demand raises `ValueError`. It validates that the queue has at most 120 entries and that each entry is finite and in $[0, 1]$. An invalid queue latches the instance invalid. These shape and range checks cannot establish that the queue corresponds to the real or simulated actuator.

[0013] For a posterior upper rise bound H , the governor sets $z_0 = H$ and propagates through every pending command:

$$z_{(j+1)} = a_{\max} z_j + b_{\max} q_j^2 + w_{\max}, \text{ for } j = 0, \dots, d-1.$$

It retains $P_q = \max(z_0, \dots, z_d)$. A candidate calculation fails if $P_q > R$. It is insufficient to inspect only the last queued state because any earlier transition could already exceed the limit. The code uses a comparison tolerance of $1e-9$.

[0014] When the queue prefix passes, the largest continuous candidate under the final transition bound is computed from

```
c = sqrt(max(0, (R-a_max z_d-w_max)/b_max)).
```

The requested candidate is restricted to $[0, \min(r, c, 1)]$. Under the validated zero-input inequality and $z_d \leq R$, the square-root numerator is nonnegative in exact arithmetic, so a zero candidate is feasible. The $\max(0, \dots)$ also guards numerical or boundary conditions.

[0015] The delivered governor chooses an action on the uniform nonnegative lattice $j \Delta$ by

```
u_star = Delta floor(min(r,c,1)/Delta).
```

The code adds $1e-12$ to the floor argument's numerator to handle representational error and clips negative outcomes to zero. It then recomputes $z_{(d+1)} = a_{\max} z_d + b_{\max} u_{\text{star}}^2 + w_{\max}$. The calculation returns true only if $\max(P_q, z_{(d+1)}) \leq R$ within the implementation tolerance. In exact arithmetic this is the largest grid member that satisfies demand, unit-effort and one-candidate bounds after the fixed queue. It is not an optimization of a multi-action future work schedule.

[0016] `admissible` returns the chosen action, the largest upper state over the current/queue/candidate prefix, and a feasibility Boolean. This returned peak concerns the **finite prefix**. It is not necessarily the maximum of the infinite zero-input tail when the terminal state is below its equilibrium. The terminal condition in [0017], rather than the displayed peak alone, supplies the tail statement.

[0017] After the queued prefix and candidate, consider applying zero at every subsequent transition. The set $[0, R]$ is invariant because the lower transition remains nonnegative and $a_{\max} R + w_{\max} \leq R$. This is equivalent to the constructor's $w_{\max}/(1-a_{\max}) \leq R$ condition. For any terminal upper state $h \leq R$, successive zero-input upper predictions are $a_{\max}^n h + w_{\max}(1-a_{\max}^n)/(1-a_{\max})$, all at most R . This tail does not assume that zero cancels the pending FIFO; zero follows all already accepted commands and the candidate.

State advancement, failures and recovery

[0018] For `interval_queue`, `Governor.decide` invokes `admissible(H, pending, r, model)`. If feasibility fails, it sets `valid=False`, stores the reason no queued fallback certificate, and returns zero requested action. That zero is a fail-closed output from the calculation; it does not itself establish that the present plant or pending queue is safe. A preexisting infeasible queue cannot be undone by appending zero.

[0019] Invalid queue data, inconsistent or invalid observations, a failed queue/candidate calculation, and invalid applied-effort acknowledgment can latch invalidity. Once `valid` is false, later decisions return zero and the stored reason without accepting later observations to restore validity. The code has no recovery method or automatic rebase. Operational reuse requires a new instance, a newly justified finite commissioning interval, and a re-established model/queue/applied-action contract; that commissioning process is external to the delivered code. Invalid demand is handled differently: it raises an exception rather than latching the instance.

[0020] `Governor.advance(acknowledged_applied)` receives an **externally asserted** applied scalar effort. A finite value in $[0, 1]$ advances both bounds using [0005]. A nonfinite or out-of-range value latches the governor invalid with reason `invalid acknowledgment` and returns without propagation. The code neither authenticates this assertion nor compares it with a physical measurement or the formerly supplied FIFO. “Acknowledged” in the variable name supplies no independent evidence of execution. Even an invalid instance can be propagated by a later valid call to `advance`; propagation does not clear its invalid latch.

[0021] A `decision_id` integer increments at the start of each `decide` call, including calls that later raise an invalid-demand exception. Returned decisions include the identifier, action, validity, reason, finite-prefix peak and posterior interval. The experiment runner logs identifiers. There is no decision-consumption table, replay rejection, message authentication, wall-clock expiration or enforceable one-use token.

Conditional argument and simulator interface

[0022] If the true state begins in the commissioned interval, the model coefficients/disturbance stay within their bounds, and each observation satisfies [0008], [0005] and [0010] preserve state containment at each update. If the FIFO and the value passed to `advance` also describe the actual applied sequence, monotonicity makes each z_j an upper bound on the corresponding future sampled state. The prefix checks followed by [0017] give the conditional sampled-temperature result for that decision. Repeated decisions preserve it when each newly accepted action is appended in the assumed order and the contract continues to hold. This is a mathematical model argument; the implementation’s finite precision remains subject to [0005]/[0010].

[0023] `run_experiments.py` implements an exact simulated FIFO. For nonzero delay it passes a copy of q to `decide`, pops the first entry as expected applied effort, appends the returned action, propagates the synthetic plant and passes expected effort to `advance`. With zero delay, the new action is the expected applied effort. The runner supplies the actual generated reading or `None`; no future disturbances or noise are passed to the policy.

[0024] The runner includes explicit out-of-contract experiments. `queue_mismatch` makes actual effort differ from the reported/expected effort during a selected interval; `model_violation` changes plant parameters outside the assumed box; `stuck_low` supplies an observation inconsistent with the nominal sensor law. These exercises do not constitute implementations that identify or repair those failures. Baseline policies `fixed_cap`, `threshold`, `nominal_queue` and `interval_no_queue` are comparators; a separately specified `interval_exact_clip` ablation deliberately tightens a clipped upper bound incorrectly. The interval/queue claim discussion addresses `interval_queue` only. A follow-up acknowledgment-mismatch experiment does not restore the contractual argument when state containment fails, even if no sampled temperature breach happens in that run.

Worked analytical example - not an experimental result

[0025] Take $T_a=25\text{ }^{\circ}\text{C}$, $C=95\text{ }^{\circ}\text{C}$, $T_{\text{limit}}=105\text{ }^{\circ}\text{C}$, $\epsilon=0.3\text{ }^{\circ}\text{C}$, $a_{\text{max}}=0.995$, $b_{\text{max}}=0.95\text{ }^{\circ}\text{C}$, $w_{\text{max}}=0.04\text{ }^{\circ}\text{C}$, and $\Delta=0.05$. Thus $R=80\text{ }^{\circ}\text{C}$ of rise and the zero-input upper equilibrium is $0.04/(1-0.995)=8\text{ }^{\circ}\text{C}$ of rise. The other model bounds may be $a_{\text{min}}=0.985$, $b_{\text{min}}=0.55$ and $w_{\text{min}}=0$. These are demonstration constants, not identified motor parameters or recommended hardware thresholds.

[0026] A prior rise interval $[60, 78]$ and clipped reading 95°C yield $[69.7, 78]$ by [0010]. The upper limit remains 78°C of rise, or 103°C absolute. It does not become 95°C absolute. Suppose three pending commands are each one and the requested effort is one. Upper predictions are $z_0=78$, $z_1=78.6$, $z_2=79.197$ and $z_3=79.791015$.

[0027] The next-command allowance is $c=\sqrt{(80-0.995*79.791015-0.04)/0.95}$, approximately 0.7732 . Downward grid selection gives $u_{\text{star}}=0.75$, and the candidate upper state is $79.966434925^\circ\text{C}$ of rise, or approximately 104.9664°C absolute. It is within the limit, and the subsequent zero tail satisfies [0017]. If a fourth full-effort command were already pending, its upper prediction would be 80.382059925 , which is above R ; the queue prefix would fail before a newly appended candidate could fix it. These values are direct arithmetic, distinct from any reported experiment table.

Figures and descriptions

[0028] **FIG. 1 - block diagram.** Sensor block **100** maps scalar plant temperature and bounded pre-clipping noise to a reading or missing-value marker. Observer **110** holds the finite rise interval and performs measurement intersection and applied-effort propagation. Queue block **120** supplies the exact accepted FIFO. Governor **130** receives the interval, FIFO and requested effort, and computes a candidate plus validity and finite-prefix peak. Adapter **140** receives the candidate, executes the existing FIFO ordering and externally supplies the applied-effort assertion. Plant **150** evolves the scalar state. Draw arrows $150 \rightarrow 100 \rightarrow 110 \rightarrow 130 \rightarrow 140 \rightarrow 150$, $140 \rightarrow 120 \rightarrow 130$, and $140 \rightarrow 110$ labeled “asserted applied effort.” Identify 100, 120, 140 and 150 as simulated/external-contract components in the delivered embodiment.

[0029] **FIG. 2 - decision flow.** Start with the commissioned model/interval. Validate demand and queue. If the instance is latched invalid, return invalid/zero. Otherwise intersect a present reading or retain the interval when missing. If inconsistent, latch invalid/zero. Predict upper states through the whole pending FIFO and reject an over-limit prefix. Compute the candidate allowance, round downward to the action grid, recompute the candidate end state and apply the prefix plus zero-terminal checks. Return the result; after the adapter’s applied-effort assertion, advance the interval for the next iteration. A separate arrow marks invalid acknowledgments as latching invalidity. Do not draw a zero-output arrow that skips or clears the FIFO.

Unimplemented alternatives

[0030] **Alternative A: multiple thermal states.** An extension could replace scalar rise by a vector of nonnegative rises for winding and housing, with $x_{\text{next}}=A x+B u^2+w$, nonnegative interval-bounded matrices, and a sensor row s measuring $T_a+s x$ before noise and clipping. A finite commissioned state set would be intersected with the corresponding slab or clipped half-space; a box representation would require a conservative outer approximation of that intersection. Queue predictions would propagate a valid set for every state component. A terminal box $[0, r]$ would require a componentwise inequality $A_{\text{max}} r+w_{\text{max}}\leq r$ under zero input, with the protected components restricted by their own thermal limits. This alternative requires a calibrated model, an appropriate representation for coupled measurement constraints, and new tests. It is not implemented or validated in the delivered source.

[0031] **Alternative B: uncertain accepted-command histories.** An extension could replace the exact FIFO with a finite set of admissible queued/applied histories under explicit bounded delay and loss assumptions. Each history would have its own compatible state set, updated only by reports whose

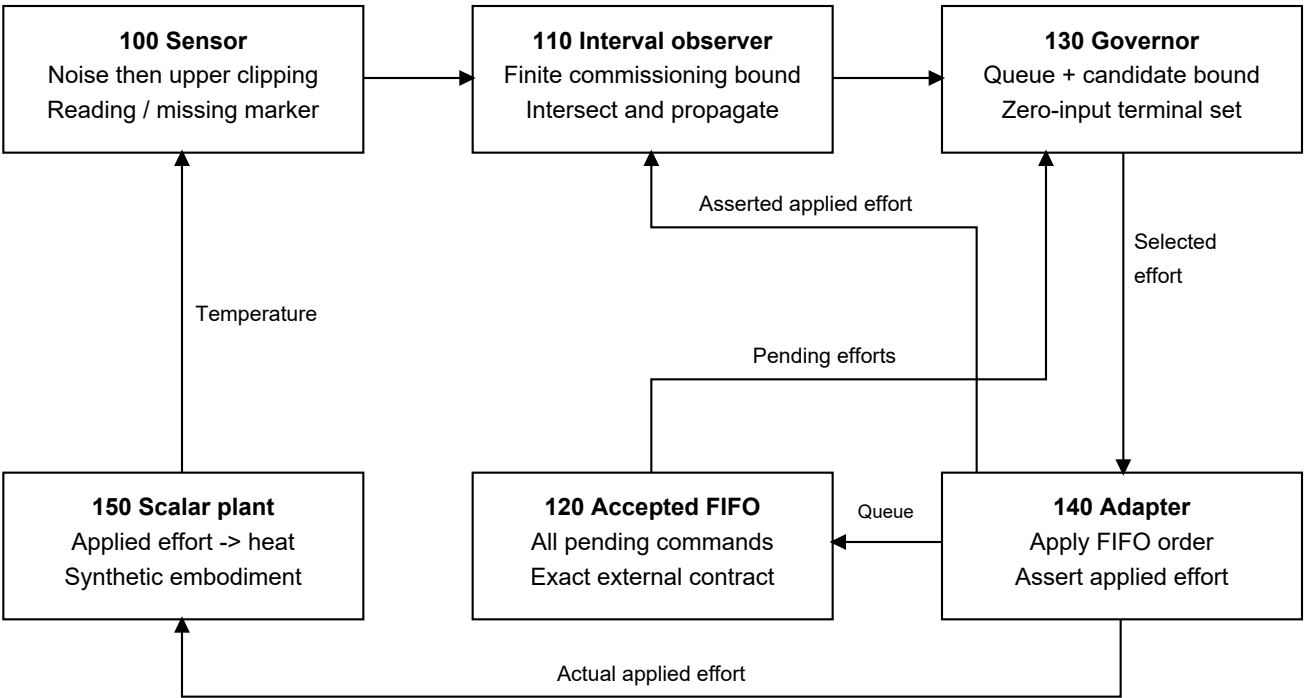
semantics identify a utilized sequence or applied transition. A proposed candidate would require a valid prefix and terminal condition for every remaining history; incompatible reports, unbounded uncertainty or an infeasible branch would prevent certification. Merging histories would need an enclosing set so that no potentially hot branch is discarded. This changes both memory/computation and the adapter contract. The delivered source has a single exact FIFO and a scalar asserted applied value; it implements none of this history inference, report validation or branch management.

Review boundary

[0032] The research evidence is in `research/prior_art_challenge.md`, references R1-R7 and P1-P4, with exact search/access records in `research/search_log.json`. R1-R4 are the nearest estimation/governor/queue/fallback challenges; P1 and R5 challenge the thermal-control setting. They are cited here by identifier to avoid repeating their summaries. The separate claim discussion supplies an internal limitation-to-paragraph/code/test matrix. Neither the description nor its alternatives establish patentability, legal inventorship, deployment readiness, or a recommendation to file.

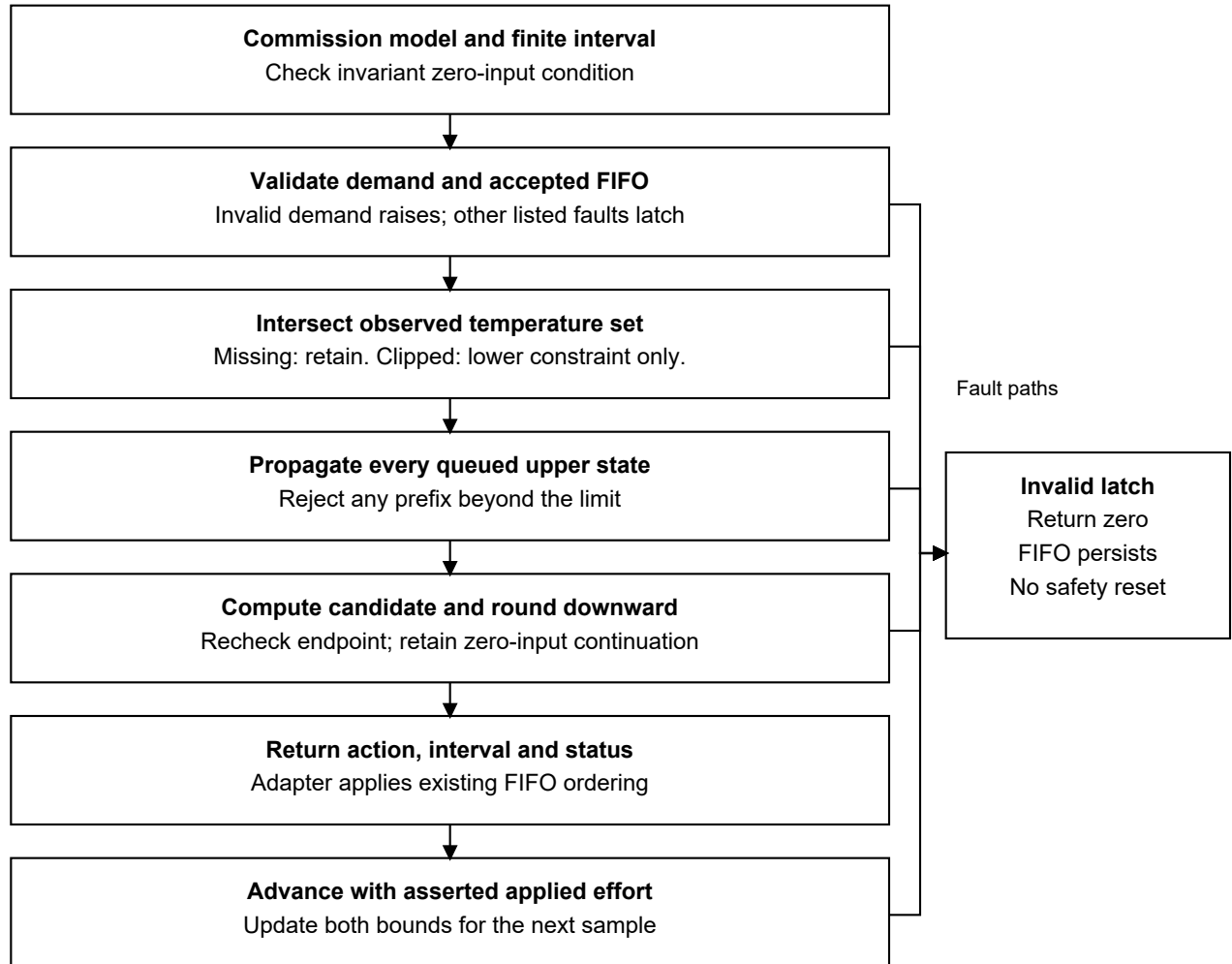
Drawings

FIG. 1. Scalar thermal governor and explicit interfaces



100, 120, 140 and 150 are simulated or externally assumed interfaces.

FIG. 2. Decision and state-advancement sequence



Internal status does not authenticate model bounds or reported actuation.